MANUAL PASO A PASO CON TDD

Dolores AR

App turística con realidad aumentada para visitar Dolores Hidalgo

GPS · Brújula · Cámara · Trigonometría · Modelos 3D · Pasaporte turístico

Desarrollo Móvil Integral · UTNG

Prof. Anastacio Rodríguez García

Antes de empezar: ¿qué vamos a construir?

Imagina que llegas a la Plaza Principal de Dolores Hidalgo, levantas tu teléfono y, sobre lo que ve la cámara, aparecen etiquetas flotando: "Parroquia de Nuestra Señora de los Dolores, 120 m", "Museo de la Independencia, 340 m", "Nieves del Jardín, 25 m". Al girar, las etiquetas se desplazan; al tocar una, aprendes su historia. Eso es realidad aumentada (RA o AR): información digital pegada al mundo real.

Dos tipos de realidad aumentada (haremos los dos)

- RA por ubicación (el corazón del proyecto): usa GPS + brújula + cámara para saber DÓNDE estás y HACIA DÓNDE miras, y dibuja etiquetas sobre los lugares. Es la técnica de apps como Pokémon GO o las guías turísticas con "monóculo". Se programa casi todo con matemáticas, así que es perfecta para TDD.
- RA de superficie (modelo 3D): coloca un objeto 3D (por ejemplo una campana de la Independencia) sobre el piso o una mesa, usando ARCore en Android y Quick Look en iPhone. Lo haremos con el paquete model_viewer_plus.

> 💡 **Dato curioso / ejemplo**
> La RA por ubicación NO necesita "visión por computadora". La cámara solo es el fondo bonito: toda la magia sale de una pregunta de trigonometría: "si estoy aquí y miro hacia allá, ¿ese lugar queda dentro de lo que veo, y en qué parte de la pantalla?". Los alumnos suelen sorprenderse de que el secreto sea geometría de secundaria.

Lo que aprenderás

- Calcular distancias y rumbos entre coordenadas GPS (fórmula de Haversine y rumbo inicial).
- Por qué los ángulos son traicioneros: el salto 359° → 1° y cómo resolverlo con TDD.
- Convertir "ángulo respecto a donde miras" en "posición en pantalla" (campo de visión).
- Suavizar la brújula con un filtro para que las etiquetas no tiemblen.
- Separar la app en capas con contratos (interfaces) para poder hacer un MODO DEMO en el salón sin estar en Dolores Hidalgo.
- Usar cámara, GPS, brújula y permisos de forma responsable (privacidad y seguridad).
- Gamificación: pasaporte turístico con sellos al visitar lugares.

Mapa del proyecto

```
dolores_ar/
  lib/
    dominio/                  <- matemáticas y reglas (Dart puro, sin plugins)
      geo.dart                <- distancia, rumbo, ángulos, campo de visión
      lugar.dart  catalogo.dart
      motor_ar.dart           <- el cerebro: ¿qué se ve y dónde?
      pasaporte.dart          <- sellos por visitar lugares
      fuentes.dart            <- contratos: FuentePosicion, FuenteBrujula
      excepciones.dart
    datos/
      fuentes_reales.dart     <- geolocator + flutter_compass
      fuentes_simuladas.dart  <- MODO DEMO (sin GPS)
    presentacion/
      ar_viewmodel.dart  fondo_camara.dart  pantalla_ar.dart
      pantalla_modelo3d.dart  pantalla_inicio.dart
```

```
  main.dart
test/
  geo_test.dart  motor_ar_test.dart  pasaporte_test.dart
  ar_viewmodel_test.dart  ayudas_geo.dart
```

El ciclo TDD de siempre

🔴 ROJO: escribe la prueba (debe FALLAR)
Escribes la prueba; falla.

🟢 VERDE: el código mínimo para pasar
Código mínimo para que pase.

🔵 REFACTOR: ordena sin romper nada
Ordenas sin romper nada.

💡 Dato curioso / ejemplo
¿Cómo pruebas una app de GPS y brújula sin salir a caminar? Esa es la pregunta central del manual. Respuesta: aislas las matemáticas y las "fuentes de datos" detrás de contratos. Así puedes simular que estás en cualquier punto del mundo mirando hacia cualquier dirección, y tus pruebas corren en milisegundos dentro de la computadora.

⚠️ Cuidado
Las coordenadas del catálogo de lugares son APROXIMADAS y de práctica. En la Ronda 9 harás un recorrido de campo para corregirlas con tu propio GPS. Es parte del aprendizaje: los datos reales siempre se verifican en el lugar.

Ronda 0 · Preparar el taller

Paso 0.1 · Crear el proyecto y agregar paquetes

```
$ flutter create dolores_ar
$ cd dolores_ar
$ flutter pub add camera geolocator flutter_compass model_viewer_plus
$ mkdir lib/dominio lib/datos lib/presentacion
```

Usar flutter pub add instala las versiones más recientes compatibles con tu Flutter. Tu pubspec.yaml debe quedar parecido a este (las versiones pueden variar):

📄 **pubspec.yaml**

```
name: dolores_ar
description: App turística con realidad aumentada para visitar Dolores Hidalgo.
publish_to: none
version: 1.0.0

environment:
  sdk: ">=3.0.0 <4.0.0"

# Si "flutter pub get" se queja de alguna versión, borra la línea y ejecuta:
#   flutter pub add camera geolocator flutter_compass model_viewer_plus
dependencies:
  flutter:
    sdk: flutter
  camera: ^0.11.0
  geolocator: ^13.0.1
  flutter_compass: ^0.8.1
  model_viewer_plus: ^1.9.3

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0

flutter:
  uses-material-design: true
```

Borra test/widget_test.dart (de ejemplo) y ejecuta flutter pub get.

Paso 0.2 · Para qué sirve cada paquete

- camera: muestra lo que ve la cámara trasera.
- geolocator: entrega tu posición GPS y gestiona el permiso de ubicación.
- flutter_compass: entrega hacia dónde apunta el teléfono (rumbo en grados).
- model_viewer_plus: muestra modelos 3D (.glb) y los lanza en realidad aumentada.

Paso 0.3 · Permisos en Android

En android/app/src/main/AndroidManifest.xml, dentro de la etiqueta manifest y ANTES de application, agrega:

📄 **android/app/src/main/AndroidManifest.xml**

```
<uses-permission android:name="android.permission.INTERNET"/>
```

```
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-feature android:name="android.hardware.camera" android:required="false"/>
```

El plugin de la cámara requiere Android 5.0 o superior. En android/app/build.gradle (o build.gradle.kts) verifica que minSdk sea 21 o más (para mejor soporte de AR, 24 o más):

```
defaultConfig {
    minSdkVersion 24
    ...
}
```

Paso 0.4 · Permisos en iPhone (si usas Mac)

En ios/Runner/Info.plist agrega los textos que verá el usuario al pedirle permiso. Escríbelos claros y honestos: iOS rechaza apps con textos vagos.

📄 **ios/Runner/Info.plist**

```
<key>NSCameraUsageDescription</key>
<string>Usamos la cámara para mostrarte los lugares históricos a tu alrededor.</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>Usamos tu ubicación solo mientras la app está abierta, para saber qué lugares tienes
cerca.</string>
```

> ⚠️ **Cuidado**
> Pedimos ubicación "solo mientras se usa la app" (WhenInUse), nunca "siempre". Principio de mínimo privilegio: pide solo el permiso que de verdad necesitas. Tu ubicación es un dato muy personal: dice dónde vives, trabajas y estudias.

Paso 0.5 · Verificar el taller

```
$ flutter test
```

> 💡 **Dato curioso / ejemplo**
> Si dice "No tests ran", todo bien: el taller está limpio. Para probar la app completa necesitarás un teléfono real (la brújula y la cámara no existen en la mayoría de los emuladores), pero por eso construiremos el MODO DEMO.

Ronda 1 · ¿A qué distancia está? (Haversine)

El problema

La Tierra es (casi) una esfera, no un plano. Si usas el teorema de Pitágoras con latitud y longitud como si fueran x e y, te equivocas: un grado de longitud mide distinto según qué tan cerca del polo estés (en el ecuador son 111 km; en Dolores Hidalgo, unos 104 km; cerca del polo, casi cero). La fórmula de Haversine calcula la distancia "por la superficie" de una esfera.

> 💡 **Dato curioso / ejemplo**
> Pitágoras en un mapa plano funciona para distancias muy cortas (como una cuadra), pero un viaje de México a Madrid te saldría con varios cientos de kilómetros de error. Haversine funciona para ambos casos, así que siempre la usamos.

Ciclo 1.1 · ROJO

Crea test/ayudas_geo.dart: un punto de origen (la Plaza Principal), una función desplazar() que mueve un punto N metros al norte y E metros al este, y lugarEn() para crear lugares de prueba. Con ellas construimos escenarios sin necesidad de GPS.

📄 **test/ayudas_geo.dart**

```
// Ayudas compartidas por las pruebas (no es una prueba en sí).
import 'dart:math' as math;

import 'package:dolores_ar/dominio/geo.dart';
import 'package:dolores_ar/dominio/lugar.dart';

const origen = Coordenada(21.1561, -100.9318);

/// Mueve un punto [norte] metros hacia el norte y [este] metros hacia el este.
Coordenada desplazar(Coordenada c, {double norte = 0, double este = 0}) {
  const metrosPorGrado = 111194.93;
  return Coordenada(
    c.lat + norte / metrosPorGrado,
    c.lon + este / (metrosPorGrado * math.cos(c.lat * math.pi / 180)),
  );
}

Lugar lugarEn(String id, Coordenada c) => Lugar(
      id: id,
      nombre: id,
      emoji: '📍',
      categoria: Categoria.plaza,
      descripcion: '',
      datoCurioso: '',
      coordenada: c,
    );
```

Ahora crea test/geo_test.dart. Empezamos con el grupo de distancia:

📄 **test/geo_test.dart (arranque + distancia)**

```
import 'package:dolores_ar/dominio/geo.dart';
import 'package:flutter_test/flutter_test.dart';
```

```
import 'ayudas_geo.dart';

void main() {
  group('distanciaMetros (Haversine)', () {
    test('un grado de latitud mide unos 111.2 km', () {
      expect(distanciaMetros(const Coordenada(0, 0), const Coordenada(1, 0)),
          closeTo(111195, 5));
    });

    test('el mismo punto está a 0 metros', () {
      expect(distanciaMetros(origen, origen), 0);
    });

    test('es simétrica: A→B = B→A', () {
      final b = desplazar(origen, norte: 300, este: 400);
      expect(distanciaMetros(origen, b), closeTo(distanciaMetros(b, origen), 0.001));
    });

    test('un punto a 300 m norte y 400 m este está a 500 m (triángulo 3-4-5)', () {
      final b = desplazar(origen, norte: 300, este: 400);
      expect(distanciaMetros(origen, b), closeTo(500, 1));
    });
  });

}
```

🔴 **ROJO: escribe la prueba (debe FALLAR)**

flutter test falla: geo.dart no existe. Las pruebas tienen el truco del triángulo 3-4-5: un punto a 300 m al norte y 400 m al este debe estar a 500 m. Es la prueba de que la fórmula "se comporta bien" en distancias cortas.

Ciclo 1.2 · VERDE

📄 **lib/dominio/geo.dart (parte 1)**

```
import 'dart:math' as math;

class Coordenada {
  final double lat;
  final double lon;
  const Coordenada(this.lat, this.lon);

  @override
  String toString() => '(${lat.toStringAsFixed(5)}, ${lon.toStringAsFixed(5)})';
}

const double _radioTierra = 6371000.0; // metros
double _rad(double grados) => grados * math.pi / 180;
double _grados(double rad) => rad * 180 / math.pi;

/// Distancia en metros entre dos puntos de la Tierra (fórmula de Haversine).
double distanciaMetros(Coordenada a, Coordenada b) {
  final dLat = _rad(b.lat - a.lat);
  final dLon = _rad(b.lon - a.lon);
  final h = math.pow(math.sin(dLat / 2), 2) +
      math.cos(_rad(a.lat)) * math.cos(_rad(b.lat)) * math.pow(math.sin(dLon / 2), 2);
  return 2 * _radioTierra * math.asin(math.sqrt(h.clamp(0.0, 1.0).toDouble()));
}
```

Lectura guiada de la fórmula

1. Se convierten las diferencias de latitud y longitud a radianes (las funciones seno y coseno trabajan en radianes, no en grados).
2. h mide "qué tan separados" están los puntos, ponderando que los meridianos se juntan hacia los polos (por eso aparece el coseno de la latitud).
3. 2 · R · asin(√h) convierte ese valor en metros usando el radio de la Tierra: 6,371,000 m.
4. clamp(0, 1) protege contra errores de redondeo: asin explota con valores como 1.0000000002.

> 💡 **Dato curioso / ejemplo**
> El nombre "haversine" viene de "half versine" (medio verseno). Los navegantes la usaban con tablas impresas siglos antes de las computadoras: una mejora enorme respecto de calcular a mano.

> 🔵 **REFACTOR: ordena sin romper nada**
> Ejecuta flutter test: las 4 pruebas de distancia pasan. Observa que Coordenada y las matemáticas viven en dominio/ y no importan NADA de Flutter.
>
> 🏆 **RETO PARA TI**
> Agrega una prueba: la distancia entre la Parroquia y la Plaza Principal del catálogo (después de la Ronda 3) debe estar entre 30 y 120 m. ¿Tiene sentido para ti caminando?

🏅 **INSIGNIA DESBLOQUEADA: Medidor de la Tierra**

Ronda 2 · ¿Hacia dónde queda? Rumbo y ángulos

Rumbo: la dirección de un lugar

El rumbo es el ángulo, medido desde el norte y girando como las manecillas del reloj, que debes seguir para llegar de A a B: norte = 0°, este = 90°, sur = 180°, oeste = 270°. Es la misma idea que los puntos cardinales de una brújula.

El enemigo: el salto 359° → 1°

Imagina que miras a 350° y un lugar está a 10°. ¿Cuánto debes girar? Si restas 10 - 350 = -340°, ¡parece que hay que dar casi una vuelta completa! Pero en realidad solo debes girar 20° a la derecha. Los ángulos "dan la vuelta": es un círculo, no una recta. Este error es el bug número uno de las apps con brújula: las etiquetas salen disparadas al lado contrario cada vez que cruzas el norte.

> 💡 **Dato curioso / ejemplo**
> Es como las manecillas del reloj: de las 11:50 a las 12:10 pasaron 20 minutos, no 11 horas y 40 minutos. Para saber "cuánto falta" usamos el camino corto del círculo.

Ciclo 2.1 · ROJO

📄 **test/geo_test.dart (agrega dentro de main)**

```
  group('rumboGrados', () {
    test('norte = 0°', () => expect(rumboGrados(origen, desplazar(origen, norte: 100)),
closeTo(0, 0.01)));
    test('este = 90°', () => expect(rumboGrados(origen, desplazar(origen, este: 100)),
closeTo(90, 0.01)));
    test('sur = 180°', () => expect(rumboGrados(origen, desplazar(origen, norte: -100)),
closeTo(180, 0.01)));
    test('oeste = 270°', () => expect(rumboGrados(origen, desplazar(origen, este: -100)),
closeTo(270, 0.01)));
    test('noreste = 45°', () => expect(rumboGrados(origen, desplazar(origen, norte: 100, este:
100)), closeTo(45, 0.05)));
  });

  group('ángulos', () {
    test('normalizar360 lleva todo a 0..360', () {
      expect(normalizar360(-90), 270);
      expect(normalizar360(360), 0);
      expect(normalizar360(725), 5);
    });

    test('diferenciaAngular toma el camino corto', () {
      expect(diferenciaAngular(10, 350), 20); // gira 20° a la derecha
      expect(diferenciaAngular(350, 10), -20); // gira 20° a la izquierda
      expect(diferenciaAngular(90, 90), 0);
      expect(diferenciaAngular(181, 0), -179);
    });

    test('suavizarAngulo avanza una fracción hacia el nuevo valor', () {
      expect(suavizarAngulo(100, 110, alfa: 0.5), closeTo(105, 1e-9));
    });

    test('suavizarAngulo cruza el 0° por el lado corto (350° → 10°)', () {
      // Un promedio ingenuo daría 180°. El correcto da 0°.
      expect(suavizarAngulo(350, 10, alfa: 0.5), closeTo(0, 1e-9));
```

```
    });

    test('nombreCardinal', () {
      expect(nombreCardinal(0), 'N');
      expect(nombreCardinal(44), 'NE');
      expect(nombreCardinal(90), 'E');
      expect(nombreCardinal(200), 'S');
      expect(nombreCardinal(270), 'O');
      expect(nombreCardinal(350), 'N');
    });
  });
```

Observa las pruebas que combinan el cruce del cero: diferenciaAngular(10, 350) debe dar 20, y suavizarAngulo(350, 10) debe dar 0°, no 180° (que sería el promedio ingenuo de 350 y 10).

Ciclo 2.2 · VERDE

📄 **lib/dominio/geo.dart (agrega)**

```
/// Rumbo (0° = norte, 90° = este, 180° = sur, 270° = oeste) para ir de [a] a [b].
double rumboGrados(Coordenada a, Coordenada b) {
  final dLon = _rad(b.lon - a.lon);
  final lat1 = _rad(a.lat);
  final lat2 = _rad(b.lat);
  final y = math.sin(dLon) * math.cos(lat2);
  final x = math.cos(lat1) * math.sin(lat2) - math.sin(lat1) * math.cos(lat2) *
math.cos(dLon);
  return normalizar360(_grados(math.atan2(y, x)));
}

/// Lleva cualquier ángulo al rango 0 <= g < 360.
double normalizar360(double g) => ((g % 360) + 360) % 360;

/// Cuánto hay que girar (en grados, de -180 a 180) para pasar de [actual] a [objetivo].
/// Positivo = girar a la derecha. Resuelve el problema del salto 359° -> 1°.
double diferenciaAngular(double objetivo, double actual) {
  var d = normalizar360(objetivo - actual);
  if (d > 180) d -= 360;
  return d;
}

/// Filtro pasa-bajas circular: suaviza la brújula sin romperse en el salto 359°/0°.
/// [alfa] cerca de 0 = muy suave y lento; cerca de 1 = rápido y tembloroso.
double suavizarAngulo(double anterior, double nuevo, {double alfa = 0.2}) =>
    normalizar360(anterior + alfa * diferenciaAngular(nuevo, anterior));
```

Lectura guiada

5. rumboGrados usa atan2 con dos ingredientes (y, x) que combinan latitudes y diferencia de longitud; el resultado sale entre -180° y 180° y lo normalizamos a 0°-360°.
6. normalizar360 usa un módulo doble: ((g % 360) + 360) % 360. En Dart, -90 % 360 ya da positivo, pero esta forma es segura en cualquier lenguaje.
7. diferenciaAngular resta, normaliza, y si el resultado pasa de 180° le resta 360°: obtiene siempre el camino corto (de -180° a 180°).
8. suavizarAngulo avanza solo una fracción (alfa) hacia el nuevo valor. Con alfa = 0.2, cada lectura mueve el valor un 20% del camino: la brújula deja de temblar.

💡 **Dato curioso / ejemplo**

Los sensores de brújula de un teléfono son muy ruidosos: pueden variar 5° o más por segundo aunque tengas el teléfono quieto (por imanes, metales o corrientes cercanos). Sin filtro, las etiquetas bailarían sin parar. El filtro es como un amortiguador de coche: absorbe los baches pequeños.

⚠️ **Cuidado**

Para suavizar ángulos NUNCA promedies los números directamente. El promedio de 350° y 10° es 180° (sur) cuando debería ser 0° (norte). Siempre promedia con la diferencia angular.

🔵 **REFACTOR: ordena sin romper nada**

Ejecuta flutter test. Las pruebas de cruce por el norte son las que justifican todo este capítulo: si alguna vez alguien "simplifica" suavizarAngulo y reintroduce el error, la prueba lo atrapa.

🏆 **RETO PARA TI**

Agrega una prueba que verifique que suavizarAngulo nunca devuelve un valor fuera de 0-360, probando con ángulos negativos y mayores que 360.

🏅 **INSIGNIA DESBLOQUEADA: Domador de ángulos**

Ronda 3 · De ángulo a pantalla: el campo de visión

La idea

Tu cámara no ve 360°: solo ve un "abanico" de unos 60° de ancho (campo de visión horizontal, FOV). Si el lugar está justo en el centro del abanico, su etiqueta va en el centro de la pantalla; si está en el borde derecho del abanico, va en el borde derecho de la pantalla; si queda fuera, no se dibuja.

```
Mirando al norte (0°) con campo de visión de 60°:

   -30°            0°            +30°
     ╱──────────────────────────╲
    /       lo que ve la cámara       \
   x = 0         x = 0.5          x = 1
 (izquierda)    (centro)        (derecha)
```

La fórmula: x = 0.5 + diferencia / campoVision. Con diferencia = 0 ⟶ 0.5; con +30° ⟶ 1.0; con -30° ⟶ 0.0. Si |diferencia| > campoVision/2 ⟶ null (no se ve).

Ciclo 3.1 · ROJO

📄 **test/geo_test.dart (agrega dentro de main)**

```
  group('posicionHorizontal (campo de visión 60°)', () {
    test('justo al frente cae en el centro', () => expect(posicionHorizontal(0), 0.5));
    test('en el borde derecho', () => expect(posicionHorizontal(30), closeTo(1.0, 1e-9)));
    test('en el borde izquierdo', () => expect(posicionHorizontal(-30), closeTo(0.0, 1e-9)));
    test('a medio camino', () => expect(posicionHorizontal(15), closeTo(0.75, 1e-9)));
    test('fuera del campo de visión no se dibuja', () {
      expect(posicionHorizontal(31), isNull);
      expect(posicionHorizontal(-90), isNull);
    });
  });

  group('formatearDistancia', () {
    test('metros y kilómetros', () {
      expect(formatearDistancia(85.4), '85 m');
      expect(formatearDistancia(999.4), '999 m');
      expect(formatearDistancia(1260), '1.3 km');
    });
  });
```

Ciclo 3.2 · VERDE

📄 **lib/dominio/geo.dart (agrega al final)**

```
/// Posición horizontal en pantalla (0 = borde izquierdo, 1 = borde derecho)
/// o null si el objeto queda fuera del campo de visión de la cámara.
double? posicionHorizontal(double diferencia, {double campoVision = 60}) {
  if (diferencia.abs() > campoVision / 2) return null;
  return 0.5 + diferencia / campoVision;
}

String formatearDistancia(double metros) =>
    metros < 1000 ? '${metros.round()} m' : '${(metros / 1000).toStringAsFixed(1)} km';

String nombreCardinal(double grados) {
  const puntos = ['N', 'NE', 'E', 'SE', 'S', 'SO', 'O', 'NO'];
```

```
  return puntos[((normalizar360(grados) + 22.5) ~/ 45) % 8];
}
```

> 💡 **Dato curioso / ejemplo**
> Devolver double? (nulo = "no se ve") es mejor que devolver un número raro como -1. El compilador de Dart te obliga a revisar el caso null antes de usar el valor, así que no puedes olvidarlo.

> ⚠️ **Cuidado**
> El campo de visión de 60° es una estimación para teléfonos típicos. Algunos tienen lentes más abiertos. Por eso el valor es un parámetro: puedes ajustarlo para tu equipo (y la prueba de "campo más ancho muestra más lugares" lo verifica).

Ciclo 3.3 · Los lugares de Dolores Hidalgo

Un Lugar es un dato sencillo: nombre, emoji, categoría, descripción, dato curioso y coordenada.

📄 **lib/dominio/lugar.dart**

```
import 'geo.dart';

enum Categoria { historia, museo, plaza, musica, gastronomia }

class Lugar {
  final String id;
  final String nombre;
  final String emoji;
  final Categoria categoria;
  final String descripcion;
  final String datoCurioso;
  final Coordenada coordenada;

  const Lugar({
    required this.id,
    required this.nombre,
    required this.emoji,
    required this.categoria,
    required this.descripcion,
    required this.datoCurioso,
    required this.coordenada,
  });
}
```

Y el catálogo con siete lugares reales de la ciudad. Recuerda: las coordenadas son aproximadas y las corregirás en campo.

📄 **lib/dominio/catalogo.dart**

```
import 'geo.dart';
import 'lugar.dart';

/// ⚠️ COORDENADAS APROXIMADAS, de práctica.
/// Tarea en campo: párate en cada lugar, lee tus coordenadas en la pantalla
/// de diagnóstico y corrige estos valores (ver Ronda 9 del manual).
const catalogoDolores = <Lugar>[
  Lugar(
    id: 'plaza',
    nombre: 'Jardín Principal',
    emoji: '🌳',
    categoria: Categoria.plaza,
```

```
    descripcion: 'El corazón de la ciudad: kioscos, árboles, vendedores y el ambiente de
siempre.',
    datoCurioso: 'Aquí se reúnen familias cada tarde, y alrededor se venden las famosas
nieves.',
    coordenada: Coordenada(21.15610, -100.93180),
  ),
  Lugar(
    id: 'parroquia',
    nombre: 'Parroquia de Nuestra Señora de los Dolores',
    emoji: '⛪',
    categoria: Categoria.historia,
    descripcion: 'Templo donde el cura Miguel Hidalgo convocó al pueblo en la madrugada del 16
de septiembre de 1810.',
    datoCurioso: 'Se le conoce como la Cuna de la Independencia Nacional.',
    coordenada: Coordenada(21.15650, -100.93150),
  ),
  Lugar(
    id: 'casa_visita',
    nombre: 'Museo Casa de Visita',
    emoji: '🏛️',
    categoria: Categoria.museo,
    descripcion: 'Antigua residencia de Miguel Hidalgo, hoy museo sobre su vida y los inicios
del movimiento.',
    datoCurioso: 'Es una de las casas más importantes de la historia de México.',
    coordenada: Coordenada(21.15590, -100.93085),
  ),
  Lugar(
    id: 'museo_independencia',
    nombre: 'Museo de la Independencia Nacional',
    emoji: '📜',
    categoria: Categoria.museo,
    descripcion: 'Exhibe documentos y objetos que cuentan cómo empezó la lucha de
Independencia.',
    datoCurioso: 'Ocupa un edificio histórico que antes tuvo otros usos.',
    coordenada: Coordenada(21.15700, -100.93230),
  ),
  Lugar(
    id: 'museo_jaj',
    nombre: 'Museo José Alfredo Jiménez',
    emoji: '🎺',
    categoria: Categoria.musica,
    descripcion: 'Homenaje al cantautor dolorense autor de "El Rey" y "Si nos dejan".',
    datoCurioso: 'José Alfredo Jiménez nació en Dolores Hidalgo en 1926.',
    coordenada: Coordenada(21.15520, -100.93260),
  ),
  Lugar(
    id: 'nieves',
    nombre: 'Nieves del Jardín',
    emoji: '🍦',
    categoria: Categoria.gastronomia,
    descripcion: 'Puestos de nieves de sabores sorprendentes: aguacate, tequila, elote,
camarón...',
    datoCurioso: 'Probar una nieve "atrevida" es casi una tradición para los visitantes.',
    coordenada: Coordenada(21.15605, -100.93165),
  ),
  Lugar(
    id: 'tumba_jaj',
    nombre: 'Tumba de José Alfredo Jiménez',
    emoji: '🌹',
    categoria: Categoria.musica,
    descripcion: 'Lugar de descanso del compositor, visitado por admiradores de todo el
país.',
    datoCurioso: 'Muchos visitantes dejan flores o canciones en su memoria.',
```

```
    coordenada: Coordenada(21.14980, -100.92700),
  ),
];
```

> 💡 **Dato curioso / ejemplo**
> Cada lugar tiene un "dato curioso" porque la gente recuerda mejor una historia que un dato seco. Una app turística gana o pierde por su contenido: los programadores y los historiadores locales hacen buen equipo.

🏅 **INSIGNIA DESBLOQUEADA: Cartógrafo de pantalla**

Ronda 4 · El motor de realidad aumentada

Juntando todo

Ya tenemos las piezas: distancia, rumbo, diferencia angular y campo de visión. El motor las combina en una sola función: dado dónde estoy, hacia dónde miro y la lista de lugares, devuelve los que se ven, con su posición horizontal, su distancia y un tamaño.

Además aplicamos tres trucos visuales que dan sensación de profundidad:

- Los lugares lejanos se dibujan más pequeños (escala de 1.2 cerca a 0.6 lejos).
- Los lugares lejanos "suben" hacia el horizonte; los cercanos bajan (profundidad).
- Se ordenan de lejos a cerca para que los cercanos se dibujen encima y tapen a los lejanos.

> 💡 **Dato curioso / ejemplo**
> Los pintores del Renacimiento descubrieron estos mismos trucos de perspectiva hace 600 años: lo lejano es más pequeño y está más cerca del horizonte. Ahora los usamos en el teléfono.

Ciclo 4.1 · ROJO

📄 **test/motor_ar_test.dart**

```
import 'package:dolores_ar/dominio/motor_ar.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas_geo.dart';

void main() {
  final frente = lugarEn('frente', desplazar(origen, norte: 100));
  final derecha = lugarEn('derecha', desplazar(origen, este: 100));
  final lejos = lugarEn('lejos', desplazar(origen, norte: 5000));

  test('un lugar justo adelante aparece en el centro de la pantalla', () {
    final r = calcularVisibles(usuario: origen, rumbo: 0, lugares: [frente]);
    expect(r.single.x, closeTo(0.5, 0.01));
    expect(r.single.distancia, closeTo(100, 1));
  });

  test('un lugar a 90° a la derecha NO aparece si miras al norte', () {
    final r = calcularVisibles(usuario: origen, rumbo: 0, lugares: [derecha]);
    expect(r, isEmpty);
  });

  test('al girar hacia el este, ese lugar aparece en el centro', () {
    final r = calcularVisibles(usuario: origen, rumbo: 90, lugares: [derecha]);
    expect(r.single.x, closeTo(0.5, 0.01));
  });

  test('si giras un poco, el lugar se desplaza al lado contrario', () {
    // Miras 10° a la derecha del norte: el lugar queda a TU izquierda.
    final r = calcularVisibles(usuario: origen, rumbo: 10, lugares: [frente]);
    expect(r.single.x, lessThan(0.5));
  });

  test('lugares más allá del radio máximo se descartan', () {
    final r = calcularVisibles(usuario: origen, rumbo: 0, lugares: [lejos], radioMax: 2000);
    expect(r, isEmpty);
  });
```

```
  test('funciona al cruzar el 0°/360° (miras a 350° y el lugar está a 5°)', () {
    final casi = lugarEn('casi', desplazar(origen, norte: 100, este: 8.7)); // ≈ 5°
    final r = calcularVisibles(usuario: origen, rumbo: 350, lugares: [casi]);
    expect(r, hasLength(1));
    expect(r.single.x, greaterThan(0.5)); // queda a la derecha
  });

  test('los lejanos van primero para dibujarse debajo de los cercanos', () {
    final cerca = lugarEn('cerca', desplazar(origen, norte: 50));
    final media = lugarEn('media', desplazar(origen, norte: 500));
    final r = calcularVisibles(usuario: origen, rumbo: 0, lugares: [cerca, media]);
    expect(r.map((v) => v.lugar.id), ['media', 'cerca']);
  });

  test('lo cercano se ve más grande que lo lejano', () {
    final cerca = lugarEn('cerca', desplazar(origen, norte: 50));
    final media = lugarEn('media', desplazar(origen, norte: 1500));
    final r = calcularVisibles(usuario: origen, rumbo: 0, lugares: [cerca, media]);
    final vCerca = r.firstWhere((v) => v.lugar.id == 'cerca');
    final vLejos = r.firstWhere((v) => v.lugar.id == 'media');
    expect(vCerca.escala, greaterThan(vLejos.escala));
    expect(vCerca.profundidad, lessThan(vLejos.profundidad));
  });

  test('un campo de visión más ancho muestra más lugares', () {
    final lado = lugarEn('lado', desplazar(origen, norte: 100, este: 80)); // ≈ 39°
    expect(calcularVisibles(usuario: origen, rumbo: 0, lugares: [lado], campoVision: 60),
isEmpty);
    expect(calcularVisibles(usuario: origen, rumbo: 0, lugares: [lado], campoVision: 100),
hasLength(1));
  });
}
```

Lee los nombres de las pruebas como si fueran las reglas del juego. Una prueba especialmente valiosa es "funciona al cruzar el 0°/360°": miras a 350° y el lugar está a 5°; debe aparecer a la derecha del centro. Sin ella, el motor parecería correcto en el 99% de las direcciones y fallaría justo al mirar al norte.

Ciclo 4.2 · VERDE

📄 **lib/dominio/motor_ar.dart**

```
import 'geo.dart';
import 'lugar.dart';

/// Un lugar ya convertido a "coordenadas de pantalla".
class LugarVisible {
  final Lugar lugar;
  final double distancia; // metros
  final double x; // 0 izquierda .. 1 derecha
  final double profundidad; // 0 = pegado a ti .. 1 = en el límite del radio
  final double escala; // 1.2 cerca .. 0.6 lejos
  const LugarVisible(this.lugar, this.distancia, this.x, this.profundidad, this.escala);
}

/// El cerebro de la realidad aumentada: dado dónde estás y hacia dónde miras,
/// ¿qué lugares aparecen en pantalla y en qué posición?
List<LugarVisible> calcularVisibles({
  required Coordenada usuario,
  required double rumbo,
  required List<Lugar> lugares,
```

```
  double campoVision = 60,
  double radioMax = 2000,
}) {
  final resultado = <LugarVisible>[];
  for (final l in lugares) {
    final d = distanciaMetros(usuario, l.coordenada);
    if (d > radioMax) continue;
    final dif = diferenciaAngular(rumboGrados(usuario, l.coordenada), rumbo);
    final x = posicionHorizontal(dif, campoVision: campoVision);
    if (x == null) continue;
    final profundidad = (d / radioMax).clamp(0.0, 1.0).toDouble();
    resultado.add(LugarVisible(l, d, x, profundidad, 1.2 - 0.6 * profundidad));
  }
  // Los lejanos primero: así los cercanos se dibujan encima.
  resultado.sort((a, b) => b.distancia.compareTo(a.distancia));
  return resultado;
}
```

```
$ flutter test
```

🔵 REFACTOR: ordena sin romper nada

El motor es una función pura: mismas entradas, mismas salidas, sin efectos secundarios. Las funciones puras son las más fáciles de probar, de entender y de reutilizar.

🏆 RETO PARA TI

Agrega al motor un parámetro "categorias" que filtre los lugares (por ejemplo, mostrar solo museos). Escribe primero la prueba.

🏅 INSIGNIA DESBLOQUEADA: Ingeniero de realidad aumentada

Ronda 5 · El pasaporte turístico (gamificación)

Una app turística se vuelve adictiva cuando el visitante tiene una meta. Creamos un pasaporte: al llegar a menos de 40 metros de un lugar, ganas su sello. Convierte un paseo en una colección, como las estampitas del álbum.

Ciclo 5.1 · ROJO

📄 **test/pasaporte_test.dart**

```
import 'package:dolores_ar/dominio/pasaporte.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas_geo.dart';

void main() {
  final plaza = lugarEn('plaza', origen);

  test('lejos de un lugar no hay sello', () {
    final p = Pasaporte();
    expect(p.registrarSiCerca(plaza, 120), false);
    expect(p.visitado('plaza'), false);
  });

  test('a menos de 40 m se gana el sello', () {
    final p = Pasaporte();
    expect(p.registrarSiCerca(plaza, 25), true);
    expect(p.visitado('plaza'), true);
    expect(p.total, 1);
  });

  test('exactamente a 40 m todavía cuenta', () {
    expect(Pasaporte().registrarSiCerca(plaza, 40), true);
  });

  test('el sello se gana una sola vez', () {
    final p = Pasaporte();
    expect(p.registrarSiCerca(plaza, 10), true);
    expect(p.registrarSiCerca(plaza, 10), false);
    expect(p.total, 1);
  });

  test('progreso es la fracción de lugares visitados', () {
    final p = Pasaporte()..registrarSiCerca(plaza, 5);
    expect(p.progreso(4), 0.25);
    expect(p.progreso(0), 0);
  });
}
```

Ciclo 5.2 · VERDE

📄 **lib/dominio/pasaporte.dart**

```
import 'lugar.dart';

/// Pasaporte turístico: ganas un sello al llegar a menos de 40 m de un lugar.
class Pasaporte {
  static const double radioSello = 40;
  final Set<String> _visitados = {};
```

```
  /// Devuelve true SOLO la primera vez que se gana el sello de ese lugar.
  bool registrarSiCerca(Lugar lugar, double distanciaMetros) {
    if (distanciaMetros > radioSello) return false;
    return _visitados.add(lugar.id);
  }

  bool visitado(String id) => _visitados.contains(id);
  int get total => _visitados.length;
  double progreso(int totalLugares) => totalLugares == 0 ? 0 : total / totalLugares;
}
```

💡 Dato curioso / ejemplo

registrarSiCerca devuelve true SOLO la primera vez gracias a Set.add(), que regresa false si el elemento ya existía. Una línea resuelve "avisa solo la primera vez": menos código, menos errores.

⚠️ Cuidado

¿Por qué 40 metros y no 5? El GPS de un teléfono tiene error de 5 a 20 metros (más cerca de edificios altos). Un radio muy pequeño frustraría al visitante parado frente al lugar sin recibir el sello.

🏅 INSIGNIA DESBLOQUEADA: Coleccionista de sellos

Ronda 6 · Fuentes de datos y el ViewModel

El truco que hace posible el modo demo

El ViewModel necesita "una fuente de posiciones" y "una fuente de rumbos". NO le importa si vienen del GPS real, de un simulador o de una prueba. Para lograrlo definimos dos contratos (clases abstractas) y las implementamos de varias formas:

```
            FuentePosicion (contrato)
            /        |            \
     PosicionGps   PosicionSimulada   PosicionFalsa
     (teléfono)    (modo demo)        (pruebas)
```

> 💡 **Dato curioso / ejemplo**
> Es como un enchufe: la lámpara (ViewModel) solo exige que el enchufe sea de dos clavijas; no sabe si la electricidad viene de la CFE, de un generador o de una batería. Cambias la fuente sin tocar la lámpara.

📄 **lib/dominio/fuentes.dart**

```
import 'geo.dart';

/// Contratos: el resto de la app NO sabe si la posición viene del GPS real
/// o de un simulador. Esa ignorancia es lo que permite probar y hacer demos.
abstract class FuentePosicion {
  Stream<Coordenada> get posiciones;
}

abstract class FuenteBrujula {
  /// Rumbo en grados (0 = norte).
  Stream<double> get rumbos;
}
```

📄 **lib/dominio/excepciones.dart**

```
abstract class ErrorDeApp implements Exception {
  const ErrorDeApp();
}

class PermisoUbicacionException extends ErrorDeApp {
  const PermisoUbicacionException();
  @override
  String toString() =>
      'Necesito el permiso de ubicación para mostrarte los lugares cercanos. Actívalo en
Ajustes.';
}

class UbicacionApagadaException extends ErrorDeApp {
  const UbicacionApagadaException();
  @override
  String toString() => 'El GPS está apagado. Actívalo para usar la realidad aumentada.';
}

class SinBrujulaException extends ErrorDeApp {
  const SinBrujulaException();
  @override
  String toString() => 'Este dispositivo no tiene brújula (magnetómetro). Usa el modo demo.';
}

/// Mensaje que SÍ puede ver el usuario (sin detalles internos).
```

```
String mensajeSeguro(Object error) =>
    error is ErrorDeApp ? error.toString() : 'Ocurrió un error inesperado. Intenta de nuevo.';
```

Ciclo 6.1 · ROJO: el ViewModel

Las pruebas usan fuentes falsas controladas con StreamController: tú decides cuándo "llega" una posición o un rumbo.

📄 **test/ar_viewmodel_test.dart**

```
import 'dart:async';

import 'package:dolores_ar/dominio/excepciones.dart';
import 'package:dolores_ar/dominio/fuentes.dart';
import 'package:dolores_ar/dominio/geo.dart';
import 'package:dolores_ar/dominio/pasaporte.dart';
import 'package:dolores_ar/presentacion/ar_viewmodel.dart';
import 'package:flutter_test/flutter_test.dart';

import 'ayudas_geo.dart';

class PosicionFalsa implements FuentePosicion {
  final c = StreamController<Coordenada>.broadcast();
  @override
  Stream<Coordenada> get posiciones => c.stream;
}

class BrujulaFalsa implements FuenteBrujula {
  final c = StreamController<double>.broadcast();
  @override
  Stream<double> get rumbos => c.stream;
}

void main() {
  late PosicionFalsa gps;
  late BrujulaFalsa brujula;
  late Pasaporte pasaporte;

  ArViewModel crear({double nortePlaza = 100}) {
    gps = PosicionFalsa();
    brujula = BrujulaFalsa();
    pasaporte = Pasaporte();
    final vm = ArViewModel(
      posicion: gps,
      brujula: brujula,
      lugares: [lugarEn('plaza', desplazar(origen, norte: nortePlaza))],
      pasaporte: pasaporte,
    );
    vm.iniciar();
    return vm;
  }

  test('mientras falte el GPS o la brújula, está esperando', () async {
    final vm = crear();
    expect(vm.esperando, true);
    gps.c.add(origen);
    await pumpEventQueue();
    expect(vm.esperando, true); // falta la brújula
    expect(vm.visibles, isEmpty);
    vm.dispose();
  });
```

```
test('con GPS y brújula calcula qué lugares se ven', () async {
  final vm = crear();
  gps.c.add(origen);
  brujula.c.add(0);
  await pumpEventQueue();
  expect(vm.esperando, false);
  expect(vm.visibles.single.lugar.id, 'plaza');
  vm.dispose();
});

test('al girar la brújula se actualizan los visibles', () async {
  final vm = crear();
  gps.c.add(origen);
  brujula.c.add(0);
  await pumpEventQueue();
  expect(vm.visibles, hasLength(1));
  // Un giro grande: la brújula se suaviza, así que mandamos varias lecturas.
  for (var i = 0; i < 40; i++) {
    brujula.c.add(180);
  }
  await pumpEventQueue();
  expect(vm.visibles, isEmpty); // ahora miras hacia el sur
  vm.dispose();
});

test('la brújula se suaviza: un salto brusco no mueve todo de golpe', () async {
  final vm = crear();
  brujula.c.add(10);
  await pumpEventQueue();
  expect(vm.rumbo, 10); // la primera lectura se acepta tal cual
  brujula.c.add(350);
  await pumpEventQueue();
  expect(vm.rumbo, closeTo(6, 0.01)); // 10 + 0.2 × (-20)
  vm.dispose();
});

test('al acercarse a un lugar gana el sello una sola vez', () async {
  final vm = crear(nortePlaza: 20);
  gps.c.add(origen);
  brujula.c.add(0);
  await pumpEventQueue();
  expect(pasaporte.visitado('plaza'), true);
  expect(vm.consumirSello()?.id, 'plaza');
  expect(vm.consumirSello(), isNull); // ya se consumió

  gps.c.add(desplazar(origen, norte: 1)); // sigue cerca: no repite
  await pumpEventQueue();
  expect(vm.ultimoSello, isNull);
  vm.dispose();
});

test('un error de permiso se muestra con mensaje amable', () async {
  final vm = crear();
  gps.c.addError(const PermisoUbicacionException());
  await pumpEventQueue();
  expect(vm.error, contains('permiso'));
  vm.dispose();
});

test('un error desconocido NO filtra detalles internos', () async {
  final vm = crear();
  brujula.c.addError(StateError('fallo interno en /sensores/mag.dart'));
```

```
    await pumpEventQueue();
    expect(vm.error, isNot(contains('/sensores')));
    expect(vm.error, contains('inesperado'));
    vm.dispose();
  });

  test('notifica a la pantalla cada vez que algo cambia', () async {
    final vm = crear();
    var avisos = 0;
    vm.addListener(() => avisos++);
    gps.c.add(origen);
    brujula.c.add(0);
    await pumpEventQueue();
    expect(avisos, greaterThanOrEqualTo(2));
    vm.dispose();
  });
}
```

Ciclo 6.2 · VERDE

📄 **lib/presentacion/ar_viewmodel.dart**

```
import 'dart:async';

import 'package:flutter/foundation.dart';

import '../dominio/excepciones.dart';
import '../dominio/fuentes.dart';
import '../dominio/geo.dart';
import '../dominio/lugar.dart';
import '../dominio/motor_ar.dart';
import '../dominio/pasaporte.dart';

class ArViewModel extends ChangeNotifier {
  final FuentePosicion _posicion;
  final FuenteBrujula _brujula;
  final List<Lugar> lugares;
  final Pasaporte pasaporte;
  final double campoVision;
  final double radioMax;

  ArViewModel({
    required FuentePosicion posicion,
    required FuenteBrujula brujula,
    required this.lugares,
    required this.pasaporte,
    this.campoVision = 60,
    this.radioMax = 2000,
  })  : _posicion = posicion,
        _brujula = brujula;

  Coordenada? usuario;
  double? rumbo;
  List<LugarVisible> visibles = [];
  String? error;
  Lugar? ultimoSello;

  StreamSubscription<Coordenada>? _subPosicion;
  StreamSubscription<double>? _subBrujula;

  bool get esperando => usuario == null || rumbo == null;
```

```
  void iniciar() {
    _subPosicion = _posicion.posiciones.listen((p) {
      usuario = p;
      _recalcular();
    }, onError: _alFallar);
    _subBrujula = _brujula.rumbos.listen((g) {
      // El primer dato se toma tal cual; los siguientes se suavizan.
      rumbo = rumbo == null ? g : suavizarAngulo(rumbo!, g);
      _recalcular();
    }, onError: _alFallar);
  }

  void _alFallar(Object e) {
    error = mensajeSeguro(e);
    notifyListeners();
  }

  void _recalcular() {
    final u = usuario;
    final r = rumbo;
    if (u != null && r != null) {
      visibles = calcularVisibles(
        usuario: u,
        rumbo: r,
        lugares: lugares,
        campoVision: campoVision,
        radioMax: radioMax,
      );
      for (final l in lugares) {
        if (pasaporte.registrarSiCerca(l, distanciaMetros(u, l.coordenada))) {
          ultimoSello = l;
        }
      }
    }
    notifyListeners();
  }

  /// La pantalla llama esto para mostrar el aviso de sello una sola vez.
  Lugar? consumirSello() {
    final l = ultimoSello;
    ultimoSello = null;
    return l;
  }

  @override
  void dispose() {
    _subPosicion?.cancel();
    _subBrujula?.cancel();
    super.dispose();
  }
}
```

```
$ flutter test
```

Lectura guiada

9. iniciar() se suscribe a los dos flujos (streams). Cada vez que llega un dato, recalcula.
10. El primer rumbo se acepta tal cual y los siguientes pasan por suavizarAngulo: la brújula se estabiliza.

11. Mientras falte la posición o el rumbo, esperando es true y la pantalla muestra "Buscando GPS y brújula…".
12. Cada vez que recalcula, revisa si ganaste algún sello. El aviso se guarda en ultimoSello y la pantalla lo consume UNA vez (consumirSello).
13. Los errores pasan por mensajeSeguro: permisos y GPS apagado muestran mensajes amables; un error desconocido NO revela detalles internos.
14. dispose() cancela las suscripciones: si no lo haces, el GPS sigue gastando batería aunque cierres la pantalla.

> ⚠️ **Cuidado**
> El GPS es de los sensores que MÁS batería consumen. Cancelar suscripciones en dispose() no es un detalle: es la diferencia entre una app que dura el día entero y una que se come la batería en una hora.

Paso 6.3 · Las implementaciones concretas

Primero las reales (se usan en el teléfono):

📄 **lib/datos/fuentes_reales.dart**

```
import 'package:flutter_compass/flutter_compass.dart';
import 'package:geolocator/geolocator.dart';

import '../dominio/excepciones.dart';
import '../dominio/fuentes.dart';
import '../dominio/geo.dart';

class PosicionGps implements FuentePosicion {
  @override
  Stream<Coordenada> get posiciones async* {
    if (!await Geolocator.isLocationServiceEnabled()) {
      throw const UbicacionApagadaException();
    }
    var permiso = await Geolocator.checkPermission();
    if (permiso == LocationPermission.denied) {
      permiso = await Geolocator.requestPermission();
    }
    if (permiso == LocationPermission.denied ||
        permiso == LocationPermission.deniedForever) {
      throw const PermisoUbicacionException();
    }
    yield* Geolocator.getPositionStream(
      locationSettings: const LocationSettings(
        accuracy: LocationAccuracy.best,
        distanceFilter: 2, // solo avisa si te moviste 2 m o más
      ),
    ).map((p) => Coordenada(p.latitude, p.longitude));
  }
}

class BrujulaReal implements FuenteBrujula {
  @override
  Stream<double> get rumbos {
    final eventos = FlutterCompass.events;
    if (eventos == null) return Stream.error(const SinBrujulaException());
    return eventos.where((e) => e.heading != null).map((e) => normalizar360(e.heading!));
  }
}
```

- Revisa PRIMERO si el GPS está encendido y luego si tienes permiso: así el mensaje de error es el correcto.
- distanceFilter: 2 hace que el GPS solo avise cuando te moviste 2 metros o más: ahorra batería y evita actualizaciones inútiles.
- Si el teléfono no tiene brújula, FlutterCompass.events es null y devolvemos un error con nombre propio (SinBrujulaException).

Y las simuladas (modo demo en el salón de clases):

📄 **lib/datos/fuentes_simuladas.dart**

```
import 'dart:async';

import '../dominio/fuentes.dart';
import '../dominio/geo.dart';

/// Modo demo: para practicar en el salón de clases sin estar en Dolores Hidalgo.
class PosicionSimulada implements FuentePosicion {
  final Coordenada punto;
  PosicionSimulada(this.punto);

  @override
  Stream<Coordenada> get posiciones => Stream<Coordenada>.value(punto);
}

class BrujulaSimulada implements FuenteBrujula {
  final _controlador = StreamController<double>.broadcast();
  double _ultimo = 0;

  @override
  Stream<double> get rumbos async* {
    yield _ultimo;
    yield* _controlador.stream;
  }

  /// Llamado por el deslizador de la pantalla demo.
  void girar(double grados) {
    _ultimo = grados;
    _controlador.add(grados);
  }

  void cerrar() => _controlador.close();
}
```

> 💡 **Dato curioso / ejemplo**
> La brújula simulada se controla con un deslizador en pantalla: lo mueves y es como si giraras sobre tu eje en la Plaza Principal. ¡Todo el salón puede ver la realidad aumentada funcionando sin salir del aula!

🏅 **INSIGNIA DESBLOQUEADA: Arquitecto de contratos**

Ronda 7 · La pantalla de realidad aumentada

Las capas de la pantalla (de atrás hacia adelante)

15. Fondo: la cámara (o un degradado si no hay cámara).
16. Etiquetas: una por cada lugar visible, posicionadas con x, profundidad y escala.
17. Panel superior: rumbo ("NE 45°"), número de lugares visibles y progreso del pasaporte.
18. Controles de demo: el deslizador (solo en modo demo).

En Flutter esto es un Stack: los widgets se apilan uno sobre otro, como hojas de acetato.

Paso 7.1 · El fondo de cámara

📄 **lib/presentacion/fondo_camara.dart**

```
import 'package:camera/camera.dart';
import 'package:flutter/material.dart';

/// Muestra la cámara trasera. Si no hay cámara o el permiso se niega
/// (emuladores, computadoras), usa un degradado: la app nunca se rompe.
class FondoCamara extends StatefulWidget {
  const FondoCamara({super.key});
  @override
  State<FondoCamara> createState() => _FondoCamaraState();
}

class _FondoCamaraState extends State<FondoCamara> {
  CameraController? _controlador;
  bool _fallo = false;

  @override
  void initState() {
    super.initState();
    _iniciar();
  }

  Future<void> _iniciar() async {
    try {
      final camaras = await availableCameras();
      final trasera = camaras.firstWhere(
        (c) => c.lensDirection == CameraLensDirection.back,
        orElse: () => camaras.first,
      );
      final c = CameraController(trasera, ResolutionPreset.medium, enableAudio: false);
      await c.initialize();
      if (!mounted) {
        await c.dispose();
        return;
      }
      setState(() => _controlador = c);
    } catch (_) {
      if (mounted) setState(() => _fallo = true);
    }
  }

  @override
  void dispose() {
    _controlador?.dispose(); // liberar la cámara es obligatorio
    super.dispose();
  }
```

```
  @override
  Widget build(BuildContext context) {
    final c = _controlador;
    if (c != null && c.value.isInitialized) {
      final t = c.value.previewSize!; // viene en horizontal: se invierte en vertical
      return SizedBox.expand(
        child: FittedBox(
          fit: BoxFit.cover,
          child: SizedBox(width: t.height, height: t.width, child: CameraPreview(c)),
        ),
      );
    }
    return Container(
      decoration: const BoxDecoration(
        gradient: LinearGradient(
          begin: Alignment.topCenter,
          end: Alignment.bottomCenter,
          colors: [Color(0xFF1B2A49), Color(0xFF4A6FA5), Color(0xFFD9A066)],
        ),
      ),
      child: _fallo
          ? const Align(
              alignment: Alignment.bottomCenter,
              child: Padding(
                padding: EdgeInsets.all(8),
                child: Text('Sin cámara: usando fondo simulado',
                    style: TextStyle(color: Colors.white70, fontSize: 12)),
              ),
            )
          : null,
    );
  }
}
```

- Si la cámara falla (emulador, permiso negado), se muestra un degradado y la app SIGUE funcionando. Una app robusta nunca se rompe por un hardware ausente.
- dispose() libera la cámara: si no la liberas, otras apps no podrán usarla.
- La vista previa viene en horizontal; la invertimos (ancho ↔ alto) y usamos FittedBox con BoxFit.cover para llenar la pantalla sin deformar.

Paso 7.2 · La pantalla de RA

📄 **lib/presentacion/pantalla_ar.dart**

```
import 'package:flutter/material.dart';

import '../dominio/geo.dart';
import '../dominio/lugar.dart';
import '../dominio/motor_ar.dart';
import 'ar_viewmodel.dart';
import 'fondo_camara.dart';

class PantallaAr extends StatefulWidget {
  /// Se recibe una "fábrica": el State crea el ViewModel UNA sola vez y es
  /// dueño de él (así Flutter puede reconstruir la ruta sin duplicarlo).
  final ArViewModel Function() crearVm;
  final Widget? controlesDemo; // el modo demo agrega aquí su deslizador
  const PantallaAr({super.key, required this.crearVm, this.controlesDemo});

  @override
```

```
  State<PantallaAr> createState() => _PantallaArState();
}

class _PantallaArState extends State<PantallaAr> {
  late final ArViewModel vm = widget.crearVm();

  @override
  void initState() {
    super.initState();
    vm.addListener(_avisarSello);
    vm.iniciar();
  }

  void _avisarSello() {
    final sello = vm.consumirSello();
    if (sello != null && mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('🏅 ¡Sello conseguido: ${sello.nombre}!')),
      );
    }
  }

  @override
  void dispose() {
    vm.removeListener(_avisarSello);
    vm.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: AnimatedBuilder(
        animation: vm,
        builder: (_, __) => Stack(fit: StackFit.expand, children: [
          const FondoCamara(),
          LayoutBuilder(builder: (_, caja) => Stack(children: [
                for (final v in vm.visibles) _etiqueta(context, v, caja),
              ])),
          SafeArea(child: _panelSuperior(vm)),
          if (widget.controlesDemo != null)
            Align(alignment: Alignment.bottomCenter, child: SafeArea(child:
widget.controlesDemo!)),
        ]),
      ),
    );
  }

  Widget _panelSuperior(ArViewModel vm) {
    final texto = vm.error ??
        (vm.esperando
            ? 'Buscando GPS y brújula…'
            : '${nombreCardinal(vm.rumbo!)} ${vm.rumbo!.round()}° · '
                '${vm.visibles.length} lugares · 🏅
${vm.pasaporte.total}/${vm.lugares.length}');
    return Align(
      alignment: Alignment.topCenter,
      child: Container(
        margin: const EdgeInsets.all(8),
        padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
        decoration: BoxDecoration(
          color: vm.error != null ? Colors.red.shade800 : Colors.black54,
          borderRadius: BorderRadius.circular(20),
```

```
        ),
        child: Text(texto, style: const TextStyle(color: Colors.white)),
      ),
    );
  }

  Widget _etiqueta(BuildContext context, LugarVisible v, BoxConstraints caja) {
    const ancho = 150.0;
    // Pseudo-perspectiva: lo lejano "sube" hacia el horizonte, lo cercano baja.
    final top = caja.maxHeight * (0.55 - 0.30 * v.profundidad);
    return Positioned(
      left: v.x * caja.maxWidth - ancho / 2,
      top: top,
      width: ancho,
      child: Transform.scale(
        scale: v.escala,
        child: GestureDetector(
          onTap: () => _detalle(context, v),
          child: Card(
            color: Colors.white.withOpacity(0.92),
            child: Padding(
              padding: const EdgeInsets.all(8),
              child: Column(mainAxisSize: MainAxisSize.min, children: [
                Text(v.lugar.emoji, style: const TextStyle(fontSize: 26)),
                Text(v.lugar.nombre,
                    textAlign: TextAlign.center,
                    maxLines: 2,
                    overflow: TextOverflow.ellipsis,
                    style: const TextStyle(fontWeight: FontWeight.bold, fontSize: 12)),
                Text(formatearDistancia(v.distancia), style: const TextStyle(fontSize: 11)),
              ]),
            ),
          ),
        ),
      ),
    );
  }

  void _detalle(BuildContext context, LugarVisible v) {
    final Lugar l = v.lugar;
    showModalBottomSheet<void>(
      context: context,
      builder: (_) => Padding(
        padding: const EdgeInsets.all(20),
        child: Column(mainAxisSize: MainAxisSize.min, crossAxisAlignment:
CrossAxisAlignment.start, children: [
          Text('${l.emoji} ${l.nombre}', style: Theme.of(context).textTheme.titleLarge),
          const SizedBox(height: 4),
          Text('A ${formatearDistancia(v.distancia)} de ti',
              style: Theme.of(context).textTheme.labelLarge),
          const SizedBox(height: 12),
          Text(l.descripcion),
          const SizedBox(height: 12),
          Text('💡 ${l.datoCurioso}'),
          const SizedBox(height: 8),
        ]),
      ),
    );
  }
}
```

Detalles de diseño

- Posición horizontal: left = x · anchoPantalla − anchoEtiqueta/2 (se resta la mitad para centrar la etiqueta sobre su punto).
- Posición vertical: top = alto · (0.55 − 0.30 · profundidad). Lo lejano sube al horizonte.
- Transform.scale aplica el tamaño según la distancia.
- Al tocar una etiqueta se abre un panel con descripción y dato curioso: la RA pasa de "bonita" a "útil".
- La pantalla recibe una "fábrica" del ViewModel (crearVm) y lo crea UNA vez en su State: así es dueña de su ciclo de vida y lo destruye al salir.

Paso 7.3 · Inicio con modo demo, y main

📄 **lib/presentacion/pantalla_inicio.dart**

```
import 'package:flutter/material.dart';

import '../datos/fuentes_reales.dart';
import '../datos/fuentes_simuladas.dart';
import '../dominio/catalogo.dart';
import '../dominio/geo.dart';
import '../dominio/pasaporte.dart';
import 'ar_viewmodel.dart';
import 'pantalla_ar.dart';
import 'pantalla_modelo3d.dart';

class PantallaInicio extends StatelessWidget {
  final Pasaporte pasaporte;
  const PantallaInicio({super.key, required this.pasaporte});

  void _irReal(BuildContext context) {
    Navigator.push(context, MaterialPageRoute<void>(
      builder: (_) => PantallaAr(
        crearVm: () => ArViewModel(
          posicion: PosicionGps(),
          brujula: BrujulaReal(),
          lugares: catalogoDolores,
          pasaporte: pasaporte,
        ),
      ),
    ));
  }

  void _irDemo(BuildContext context) {
    // Te "teletransporta" a la Plaza Principal para practicar en el salón.
    final brujula = BrujulaSimulada();
    Navigator.push(context, MaterialPageRoute<void>(
      builder: (_) => PantallaAr(
        crearVm: () => ArViewModel(
          posicion: PosicionSimulada(const Coordenada(21.15600, -100.93175)),
          brujula: brujula,
          lugares: catalogoDolores,
          pasaporte: pasaporte,
        ),
        controlesDemo: _DeslizadorRumbo(brujula: brujula),
      ),
    ));
  }

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: AppBar(title: const Text('Dolores AR')),
      body: ListView(padding: const EdgeInsets.all(16), children: [
        Text('Descubre Dolores Hidalgo con realidad aumentada',
            style: Theme.of(context).textTheme.headlineSmall),
        const SizedBox(height: 8),
        const Text('Apunta tu teléfono y mira qué lugares históricos te rodean.'),
        const SizedBox(height: 20),
        FilledButton.icon(
          onPressed: () => _irReal(context),
          icon: const Icon(Icons.explore),
          label: const Text('Explorar con GPS real'),
        ),
        const SizedBox(height: 10),
        OutlinedButton.icon(
          onPressed: () => _irDemo(context),
          icon: const Icon(Icons.school),
          label: const Text('Modo demo (salón de clases)'),
        ),
        const SizedBox(height: 10),
        OutlinedButton.icon(
          onPressed: () => Navigator.push(context,
              MaterialPageRoute<void>(builder: (_) => const PantallaModelo3D())),
          icon: const Icon(Icons.view_in_ar),
          label: const Text('Modelo 3D en tu espacio'),
        ),
        const SizedBox(height: 24),
        Text('Lugares del recorrido', style: Theme.of(context).textTheme.titleMedium),
        for (final l in catalogoDolores)
          ListTile(
            leading: Text(l.emoji, style: const TextStyle(fontSize: 26)),
            title: Text(l.nombre),
            trailing: Icon(pasaporte.visitado(l.id) ? Icons.verified : Icons.circle_outlined),
          ),
      ]),
    );
  }
}

class _DeslizadorRumbo extends StatefulWidget {
  final BrujulaSimulada brujula;
  const _DeslizadorRumbo({required this.brujula});
  @override
  State<_DeslizadorRumbo> createState() => _DeslizadorRumboState();
}

class _DeslizadorRumboState extends State<_DeslizadorRumbo> {
  double _grados = 0;
  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.black54,
      padding: const EdgeInsets.symmetric(horizontal: 12),
      child: Row(children: [
        const Text('Gira:', style: TextStyle(color: Colors.white)),
        Expanded(
          child: Slider(
            min: 0,
            max: 359,
            value: _grados,
            onChanged: (g) {
              setState(() => _grados = g);
              widget.brujula.girar(g);
```

```
            },
          ),
        ),
        Text('${_grados.round()}°', style: const TextStyle(color: Colors.white)),
      ]),
    );
  }
}
```

📄 **lib/main.dart**

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'dominio/pasaporte.dart';
import 'presentacion/pantalla_inicio.dart';

void main() {
  WidgetsFlutterBinding.ensureInitialized();
  // La brújula y la cámara se comportan mejor con el teléfono en vertical.
  SystemChrome.setPreferredOrientations([DeviceOrientation.portraitUp]);
  runApp(MaterialApp(
    title: 'Dolores AR',
    theme: ThemeData(colorSchemeSeed: Colors.deepOrange, useMaterial3: true),
    home: PantallaInicio(pasaporte: Pasaporte()),
  ));
}
```

> 💡 **Dato curioso / ejemplo**
> Bloqueamos la orientación en vertical a propósito: la brújula del teléfono mide hacia dónde apunta el dispositivo y gira de forma distinta en horizontal. Una app de RA debe decidir una postura y cuidarla.

Paso 7.4 · Primera prueba: el modo demo

```
$ flutter run
```

19. Toca "Modo demo (salón de clases)". Estás "teletransportado" a la Plaza Principal.
20. Mueve el deslizador "Gira": verás aparecer y desaparecer etiquetas de los lugares.
21. Gira hasta que el rumbo marque N, E, S y O: ¿qué lugares aparecen en cada dirección?
22. Toca una etiqueta: se abre el detalle.
23. Gana tu primer sello: en el modo demo estás a pocos metros del Jardín Principal y de las Nieves. Observa el aviso "🏅 Sello conseguido".
24. Regresa al inicio: la lista de lugares marca con una palomita los que ya visitaste.

> 🏆 **RETO PARA TI**
> El deslizador gira de 0° a 359°. Cambia la posición simulada a otro punto (por ejemplo, cerca del Museo José Alfredo Jiménez) y observa cómo cambian las etiquetas.

🏅 **INSIGNIA DESBLOQUEADA: Mago de la realidad aumentada**

Ronda 8 · RA de superficie: modelo 3D en tu espacio

¿Cómo funciona?

Aquí la RA es distinta: el teléfono reconoce el piso o una mesa (con ayuda de ARCore en Android y ARKit en iPhone) y coloca un modelo 3D sobre ella, a escala real, como si estuviera ahí. Puedes caminar alrededor. Para un turista: ver la campana de la Independencia del tamaño real en su sala.

Paso 8.1 · La pantalla

📄 **lib/presentacion/pantalla_modelo3d.dart**

```
import 'package:flutter/material.dart';
import 'package:model_viewer_plus/model_viewer_plus.dart';

/// AR de superficie: el visitante coloca un modelo 3D sobre el piso o una mesa.
/// En Android abre Scene Viewer (ARCore); en iPhone, Quick Look (necesita .usdz).
class PantallaModelo3D extends StatelessWidget {
  /// Modelo de práctica (astronauta). Cámbialo por TU modelo, por ejemplo una
  /// campana o el monumento a Hidalgo creado con fotogrametría:
  ///   src: 'assets/modelos/campana.glb'   (y decláralo en pubspec.yaml)
  static const modeloDePractica =
      'https://modelviewer.dev/shared-assets/models/Astronaut.glb';

  const PantallaModelo3D({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Modelo 3D en tu espacio')),
      body: const ModelViewer(
        src: modeloDePractica,
        alt: 'Modelo 3D de práctica',
        ar: true,
        arModes: ['scene-viewer', 'webxr', 'quick-look'],
        autoRotate: true,
        cameraControls: true,
      ),
    );
  }
}
```

- src: el archivo del modelo en formato .glb (un formato 3D compacto, como el JPG de los modelos 3D).
- ar: true y arModes: indican con qué tecnologías lanzar la RA: scene-viewer (Android/ARCore), quick-look (iPhone) o webxr.
- autoRotate y cameraControls: antes de lanzar la RA, el usuario puede girar el modelo con el dedo.

> ⚠️ **Cuidado**
> El modelo de práctica (un astronauta) se descarga de internet cada vez. Para tu app final descarga o crea tu propio modelo, guárdalo en assets/modelos/, decláralo en pubspec.yaml y usa src: "assets/modelos/campana.glb". En iPhone, Quick Look necesita además una versión .usdz (parámetro iosSrc). Revisa el README del paquete model_viewer_plus: los requisitos de cada plataforma pueden cambiar.

Paso 8.2 · ¿De dónde saco un modelo 3D propio?

- Fotogrametría: fotografías de un objeto desde muchos ángulos y una app (como Polycam o Reality Capture) las convierte en modelo 3D. ¡Un buen proyecto de equipo en la plaza!
- Modelado manual: Blender (gratuito) permite crear y exportar a .glb.
- Bibliotecas con licencia libre: busca modelos con licencia Creative Commons y respeta la atribución.

> 💡 **Dato curioso / ejemplo**
> Una campana moderna escaneada con fotogrametría puede pesar más de 50 MB. Para un teléfono, optimiza: menos polígonos y texturas más pequeñas. Un modelo que tarda 30 segundos en cargar es un modelo que nadie ve.

> ⚠️ **Cuidado**
> Respeta los derechos: fotografiar monumentos públicos suele estar permitido, pero verifica las reglas del museo o el recinto, y nunca uses modelos de otros sin permiso o licencia.

> 🏆 **RETO PARA TI**
> Fotografía (con permiso) un objeto pequeño de tu casa, conviértelo en .glb con una app de fotogrametría y reemplaza el astronauta por tu modelo.

🏅 **INSIGNIA DESBLOQUEADA: Escultor digital**

Ronda 9 · Prueba de campo, privacidad y seguridad

9.1 · Calibrar el catálogo (tarea de campo)

Las coordenadas del catálogo son aproximadas. Un recorrido de campo en equipo las corrige:

25. Instala la app en un teléfono real con GPS.
26. Agrega temporalmente una línea de diagnóstico en el panel superior que muestre las coordenadas actuales (usuario.toString() ya imprime 5 decimales).
27. Párate en la puerta de cada lugar, espera 30 segundos a que el GPS se estabilice y anota las coordenadas.
28. Actualiza catalogo.dart con los valores medidos.
29. Compara: ¿el lugar aparece de frente cuando lo miras? Si aparece desplazado, revisa tu rumbo y tu campo de visión.

> 💡 **Dato curioso / ejemplo**
> Un GPS de teléfono tiene un error típico de 3 a 10 metros al aire libre, y puede subir a 20 o 30 m entre edificios altos o bajo techo. Por eso las etiquetas a veces "flotan" un poco y por eso el radio del sello es de 40 m.

9.2 · Calibrar la brújula

- Si las etiquetas aparecen desplazadas hacia un lado, la brújula probablemente necesita calibración. Mueve el teléfono dibujando un "8" en el aire varias veces.
- Aléjate de objetos metálicos, imanes y fundas magnéticas: alteran el campo magnético.
- Mantén el teléfono casi vertical y sin inclinarlo mucho hacia arriba o abajo: inclinarlo cambia la lectura.
- Ajusta el campoVision (por ejemplo, 55 o 70) hasta que las etiquetas coincidan con los lugares reales.

9.3 · Tabla de problemas comunes

- "Buscando GPS y brújula…" no termina → permiso negado, GPS apagado o estás bajo techo. Sal a cielo abierto.
- Etiquetas que saltan o tiemblan → baja alfa en suavizarAngulo (por ejemplo, 0.1) o aléjate de fuentes magnéticas.
- Las etiquetas aparecen en el lado contrario → revisa que no hayas invertido el signo en diferenciaAngular (la prueba de cruce por el norte lo atrapa).
- La cámara se ve negra → falta permiso de cámara, o estás en un emulador (usa el degradado o un teléfono real).
- Se acaba la batería muy rápido → cancela suscripciones en dispose() y sube distanceFilter.

9.4 · Privacidad: tu ubicación es un dato sensible

- Pide solo el permiso necesario: ubicación "mientras se usa la app". Nunca "siempre" si no lo necesitas.
- Todo el cálculo ocurre DENTRO del teléfono: esta app no envía tu ubicación a ningún servidor. Si algún día agregas un servidor, avisa y pide consentimiento explícito.
- Escribe un aviso de privacidad claro y corto: qué datos se usan, para qué, y que no se comparten.

- Si guardas el pasaporte (reto), no guardes coordenadas exactas del usuario: solo los ids de lugares visitados.
- Maneja los rechazos con respeto: si el usuario niega el permiso, explica por qué lo necesitas y ofrece el modo demo.

9.5 · Seguridad física y responsabilidad

- Una app de realidad aumentada invita a caminar mirando la pantalla. Agrega un aviso al iniciar: "Mira a tu alrededor, cuida tu paso y no cruces la calle viendo el teléfono".
- Diseña pensando en el entorno: las etiquetas deben ser legibles al sol (alto contraste) y no tapar toda la vista.
- Ofrece alternativas accesibles: la lista de lugares del inicio funciona sin cámara para personas que no pueden o no quieren usar RA.

> 💡 **Dato curioso / ejemplo**
> Tras el lanzamiento de Pokémon GO en 2016 se reportaron numerosos casos de personas distraídas por el teléfono mientras caminaban. Desde entonces, los juegos de RA incluyen advertencias de seguridad: la seguridad es una función de diseño, no un adorno.

🏅 **INSIGNIA DESBLOQUEADA: Explorador responsable**

Cierre · Lo que lograste

Checklist de calidad

- Toda la matemática vive en dominio/ y no importa Flutter ni plugins.
- Hay pruebas para cada regla: distancia, rumbo, cruce del norte, campo de visión, ordenamiento, sellos, errores.
- Las fuentes de GPS y brújula son contratos: se pueden simular y probar.
- Existe un modo demo que funciona sin GPS ni brújula.
- La app no se rompe sin cámara, sin permiso o sin brújula.
- Se cancelan las suscripciones y se libera la cámara en dispose().
- Se pide el permiso mínimo y los textos de permiso son claros.
- Los errores inesperados muestran un mensaje genérico (no filtran detalles).
- Hay aviso de seguridad física y alternativa sin cámara.

Preguntas para discutir en clase

30. ¿Por qué la distancia entre dos puntos GPS no se calcula con el teorema de Pitágoras?
31. Miras a 350° y un lugar está a 15°. ¿Cuánto debes girar y hacia dónde? ¿Y si miras a 15° y el lugar está a 350°?
32. ¿Por qué suavizar la brújula con un promedio normal de números produciría un bug al mirar al norte?
33. ¿Qué ventaja tiene que el ViewModel dependa de FuentePosicion y no de Geolocator?
34. ¿Qué datos de la app son sensibles? ¿Cómo los proteges?
35. ¿Qué pasaría si el campo de visión real de tu cámara fuera de 80° y tu app asume 60°?

Retos finales por equipos

- Guía de audio: al tocar un lugar, que un narrador lo lea en voz alta (paquete flutter_tts).
- Persistencia: guarda el pasaporte con shared_preferences y escribe primero las pruebas con un doble.
- Contenido en línea: trae la descripción de cada lugar desde la API de Wikipedia (usa el manual de las 5 APIs) con caché para funcionar sin internet.
- Ruta guiada: una flecha que apunte al siguiente lugar sin visitar usando rumboGrados y diferenciaAngular.
- RA con anclas reales: investiga ARCore y ARKit (paquetes como ar_flutter_plugin) para fijar objetos 3D al mundo.
- Versión en inglés para turistas extranjeros usando los archivos de internacionalización de Flutter.
- Mini mapa: agrega un radar circular que muestre los lugares alrededor según rumbo y distancia.

🏅 **INSIGNIA DESBLOQUEADA: Creador de experiencias de realidad aumentada**

Recuerda: la mejor realidad aumentada no reemplaza la mirada, la enriquece. Que tu app invite a levantar la vista y conocer Dolores Hidalgo. ¡Éxito en tu recorrido!